Bilkent University

Department of Electrical and Electronics Engineering

Lab 6 Report: Arbitrary Waveform Generator

Purpose

The main goal of this experiment is to design and implement an arbitrary digital waveform generator using VHDL. To get a clean and stable clock signal, Clocking Wizard IP was utilized. The generated waveform is then observed and verified on an oscilloscope.

Design Specifications

According to the lab manual, the design must be implemented using the block design feature in Vivado. First I wrote a VHDL module named arbitrary_waveform that handles the core logic. Since the Basys3 board has a 100 MHz internal clock, I used a Clocking Wizard IP to generate a 50 MHz clock (clk_50MHz) to use in my logic.

The waveform needs to be 80 µs high, 50 µs low, 70 µs high and 50 µs low repeatedly. My VHDL code uses a finite-state machine (FSM) with four states to track this sequence. Since my operating clock is 50 MHz, the clock period is 20 ns. I calculated the necessary counter limits as follows:

- 80 µs / 20 ns = 4000 clock cycles
- 50 µs / 20 ns = 2500 clock cycles
- 70 µs / 20 ns = 3500 clock cycles

The counter checks these constants and changes the wave_out signal accordingly. Everything runs under a single process synchronized to the 50MHz clock to avoid any timing issues.

Methodology

Unlike the single-file approach, I followed the block design rule for this experiment. First, I created the VHDL file for the waveform logic. It only takes a 50 MHz clock and a reset input, and gives the digital waveform as output. Then I opened the IP Integrator and created a block design. I added the Clocking Wizard IP and configured its output to 50 MHz. After that I added my custom VHDL module to the schematic.

Using the connection automation, I wired the 100 MHz system clock and the reset button (U18) to the IP. I connected the 50 MHz output of the IP directly to the clock input

of my waveform module. The wave_out pin was made external so it can be routed to the PMOD J1 pin. After finishing the wiring, I created an HDL wrapper for the design.

Before programming the board, I wrote a testbench for the wrapper file to simulate the behavior. I set the clock period to 10 ns to simulate the 100MHz source. After verifying the timestamps in the simulation, I generated the bitstream and programmed the Basys3. For physical measurements I connected the oscilloscope probe to the JA PMOD connector's J1 pin, and attached the ground clip to the GND pin of the board.

Results

Since the design was implemented using the IP Integrator, the overall structure of the system is best represented by the block design layout. Figure 1 shows the top level connections between the 100MHz input clock, the Clocking Wizard IP, and my arbitrary waveform module.

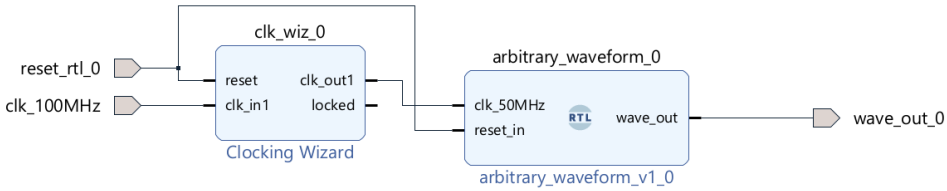


Figure 1: Vivado IP Integrator Block Design of the system

To see the underlying logic and internal connections generated by Vivado, the normal RTL schematic and the elaborated schematic are provided below. These schematics show how the IP and my VHDL code are translated into actual hardware components.

Because I used the block design tool, the structure of the system can be seen very clearly. The normal RTL schematic and the elaborated schematic are provided below to show how the IP and my module are connected.

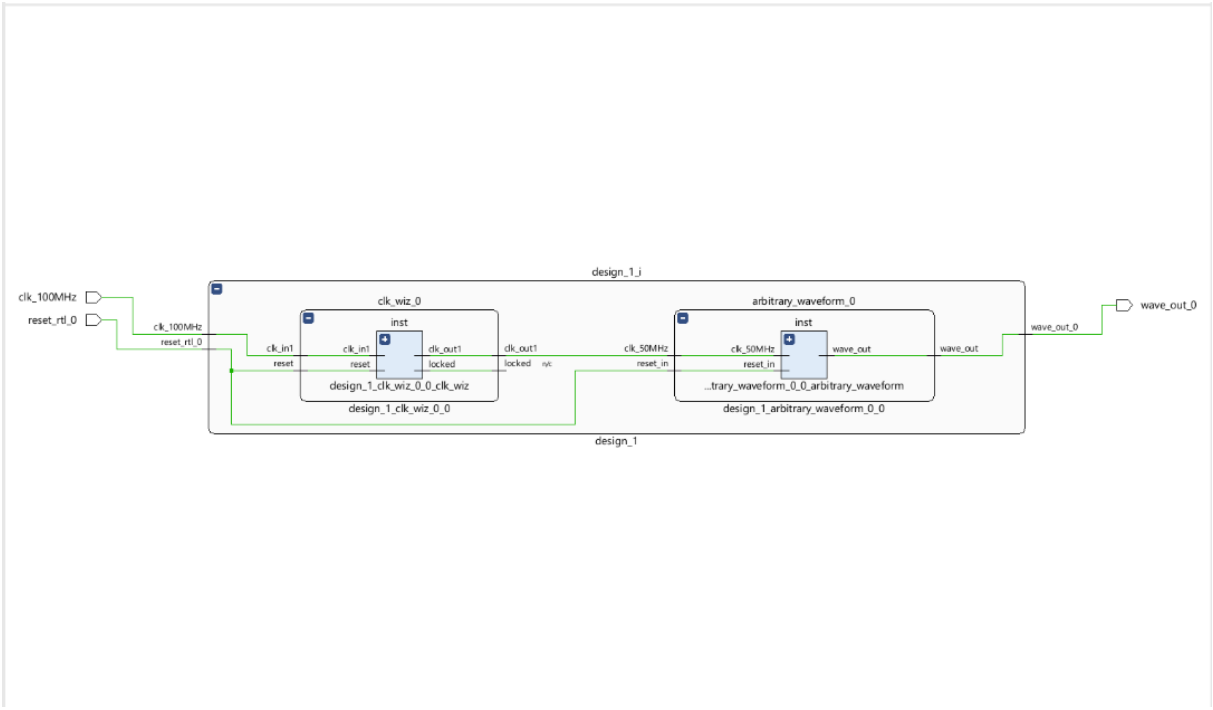


Figure 2: RTL Schematic of the Waveform Design

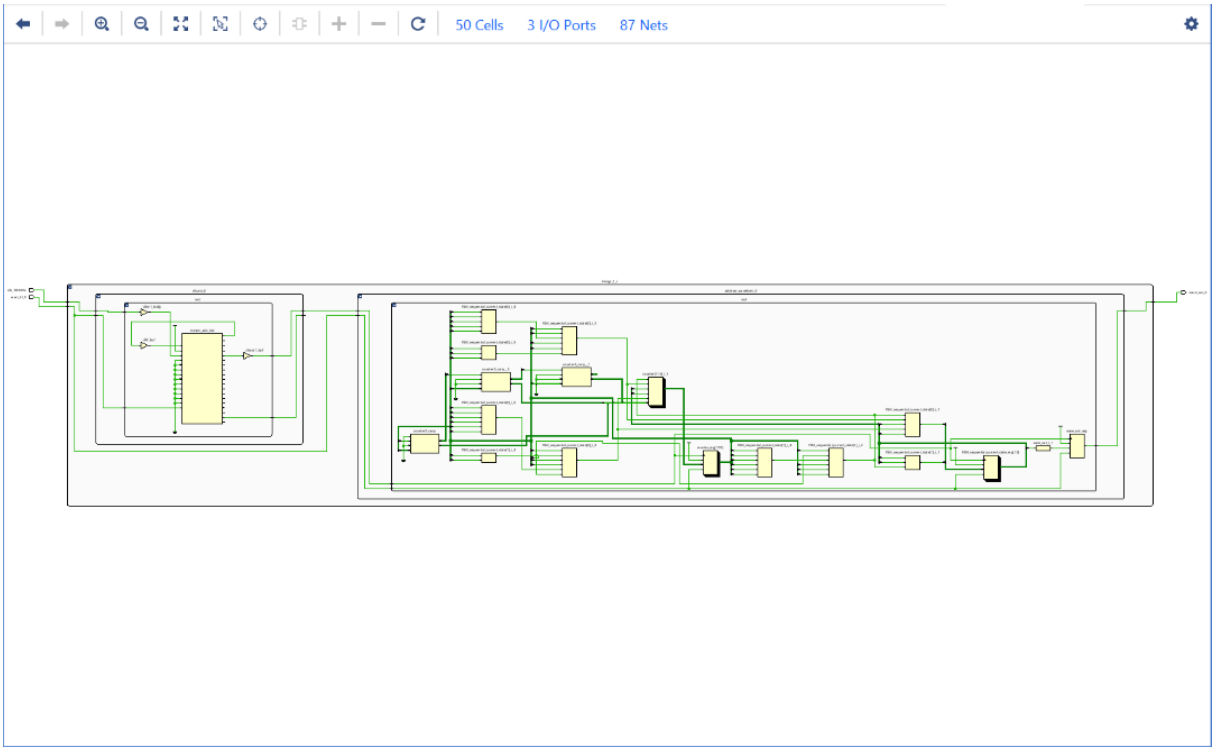


Figure 3: Elaborated Design Schematic showing internal logic Gates

To check if the logic works correctly I ran a behavioral simulation. I used markers to measure the time intervals of the high and low states. As seen in the figures below, the time difference between the rising and falling edges exactly match 80 µs, 50 µs and 70 µs.

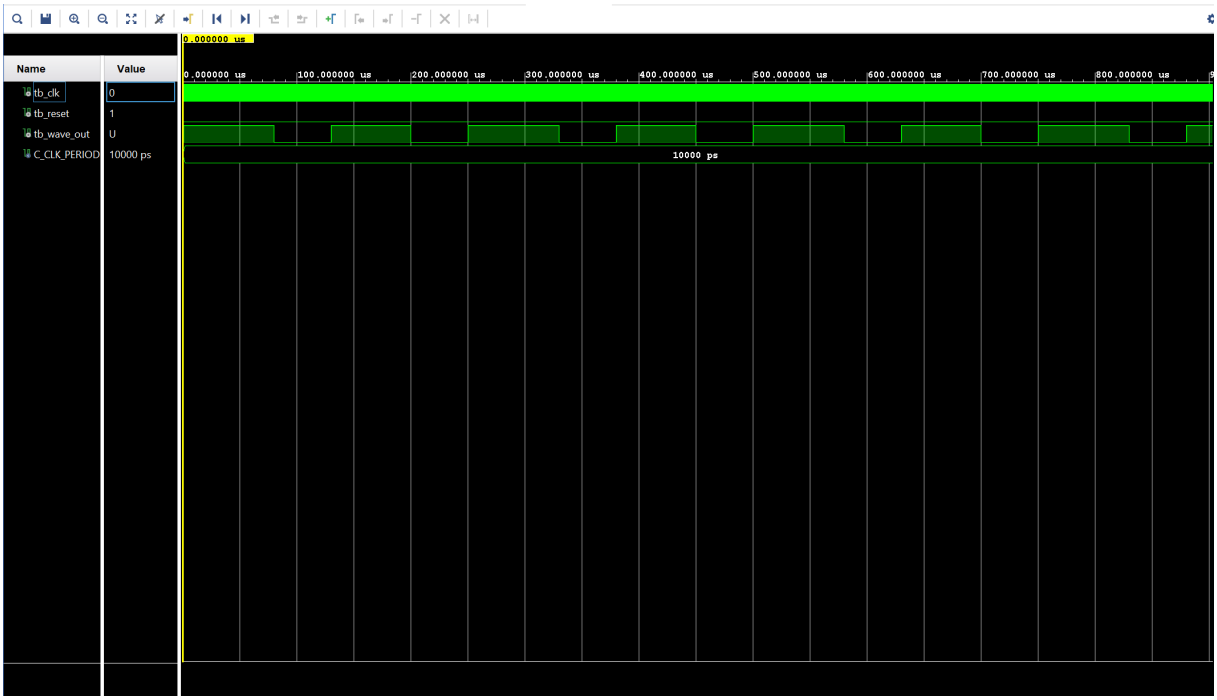


Figure 4: Simulation Results of the Waveform Design

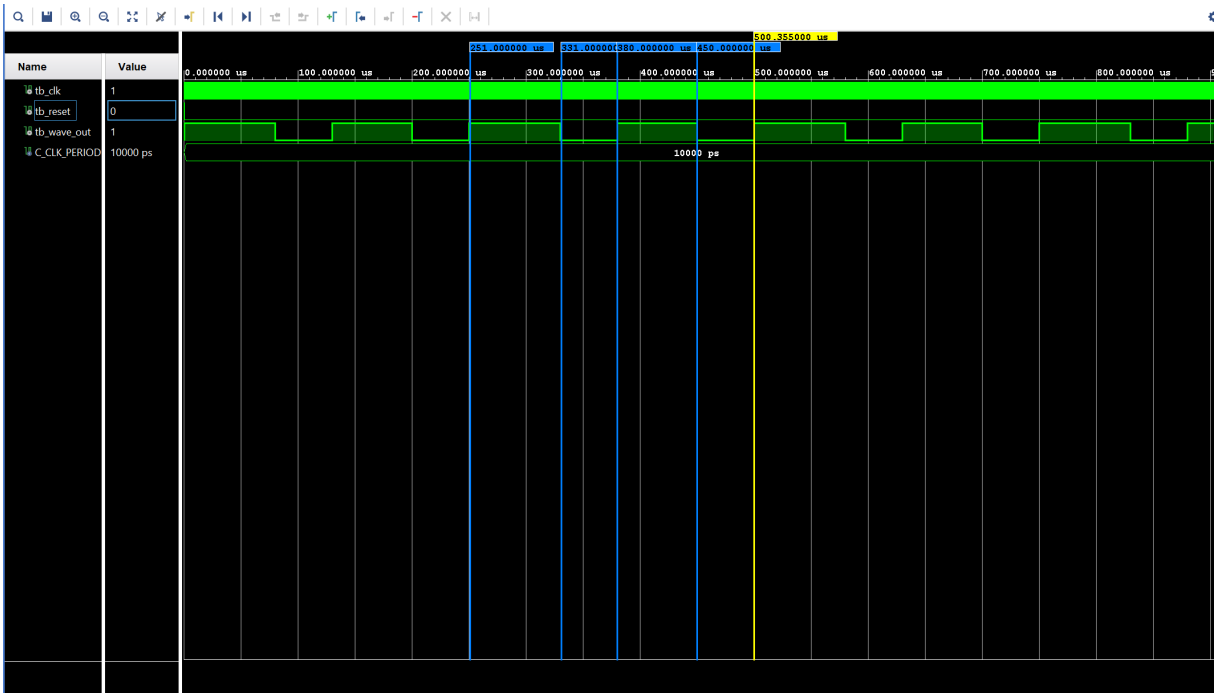


Figure 5: Simulation Result of the Waveform including Timestamps

After simulation, the design was tested on the hardware. The physical connection to the Basys3 board is shown below. It is important to connect the ground clip to get a clean signal.

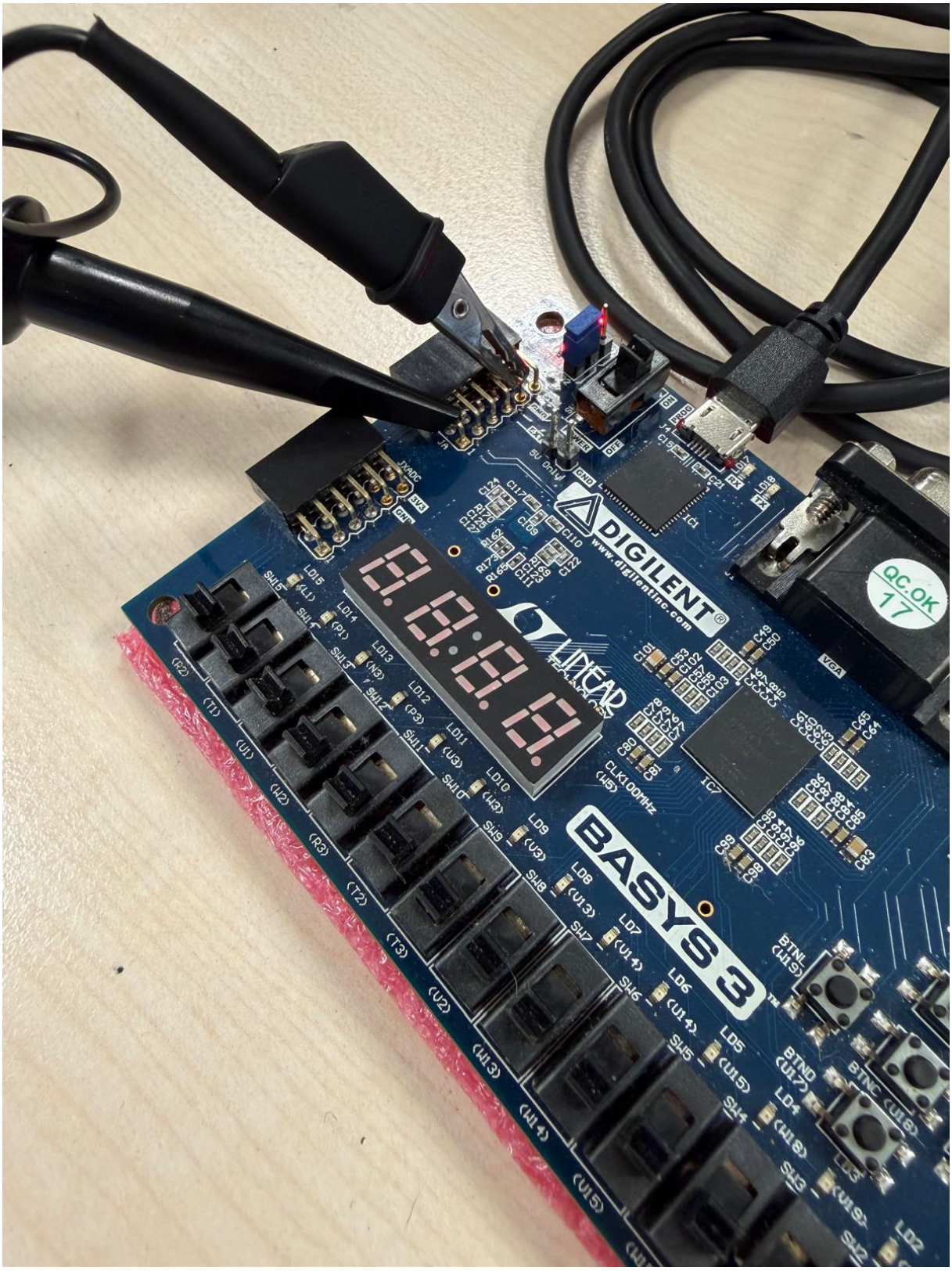


Figure 6: Oscilloscope probe connected to PMOD JA (pin J1 and GND)

I used the Keysight oscilloscope's cursor function to measure the real life time intervals of the generated signal. The measurements perfectly matched my simulation results and calculations.

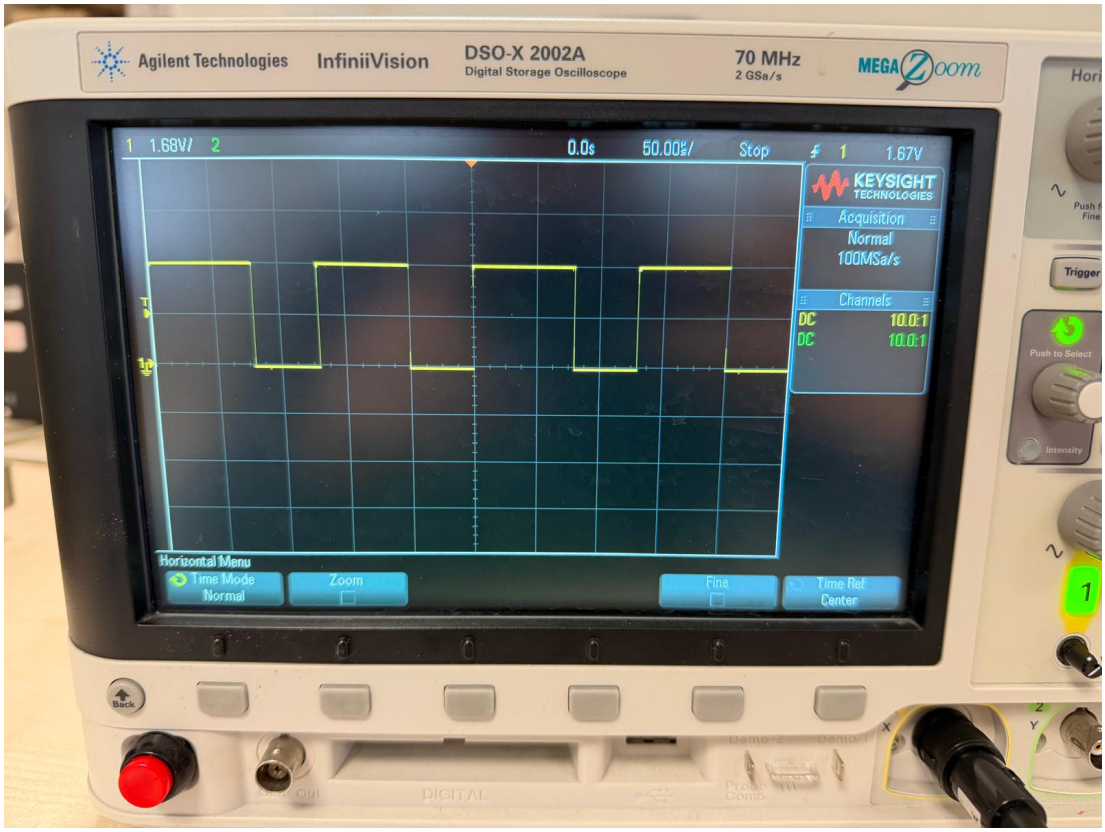


Figure 7: General Behavior of Arbitrary Waveform on Oscilloscope

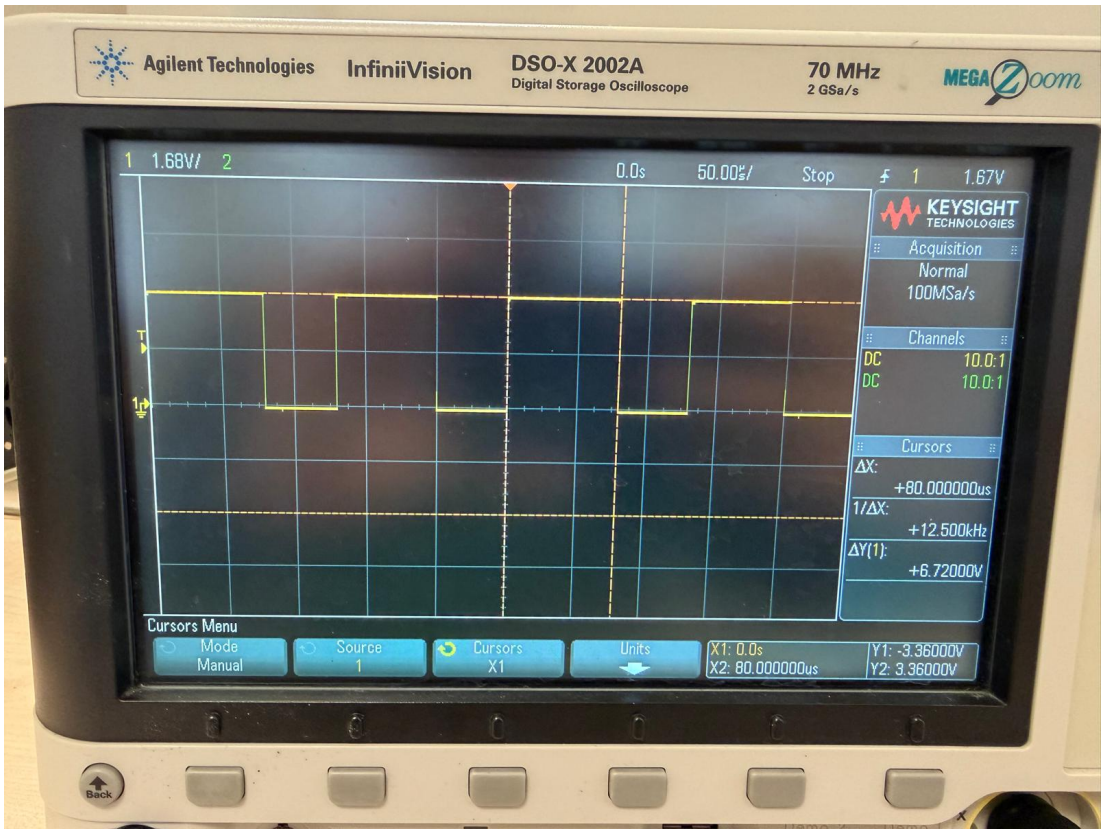


Figure 8: 80 µs Logic High Wave measured on oscilloscope

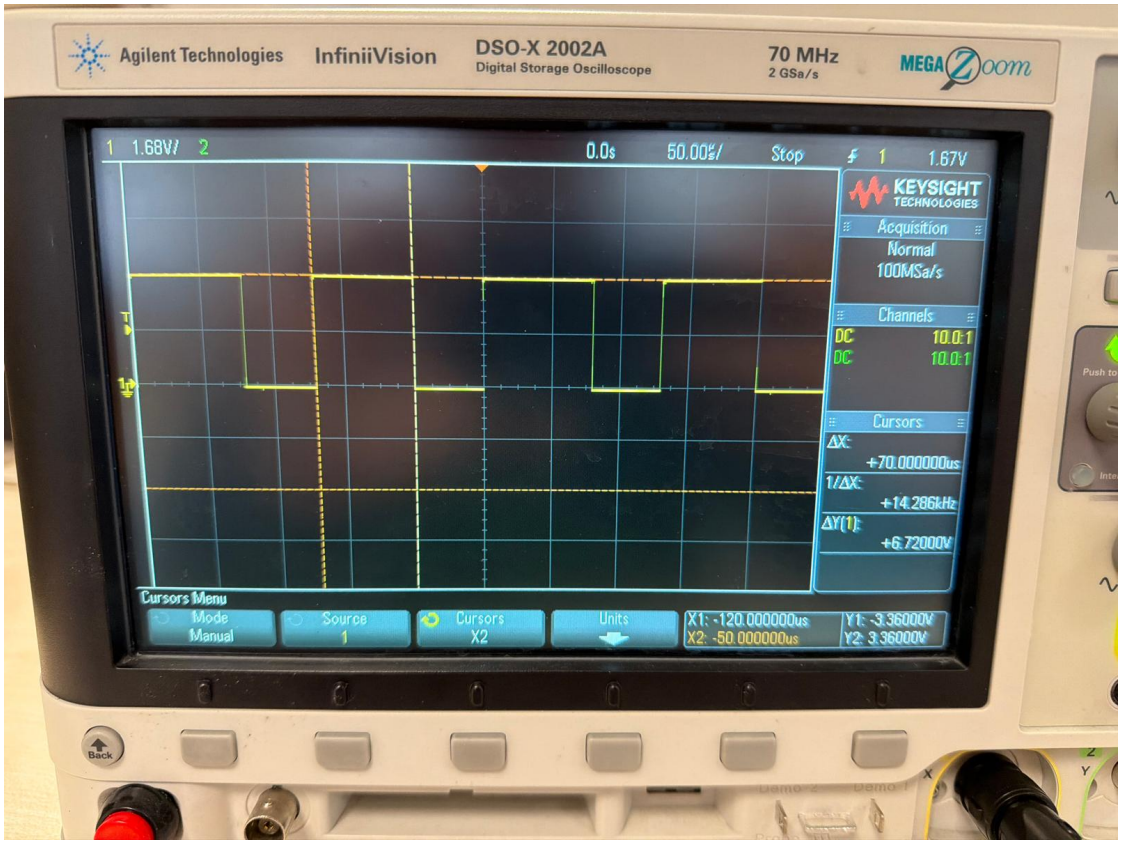


Figure 9: 70 µs Logic High Wave measured on oscilloscope

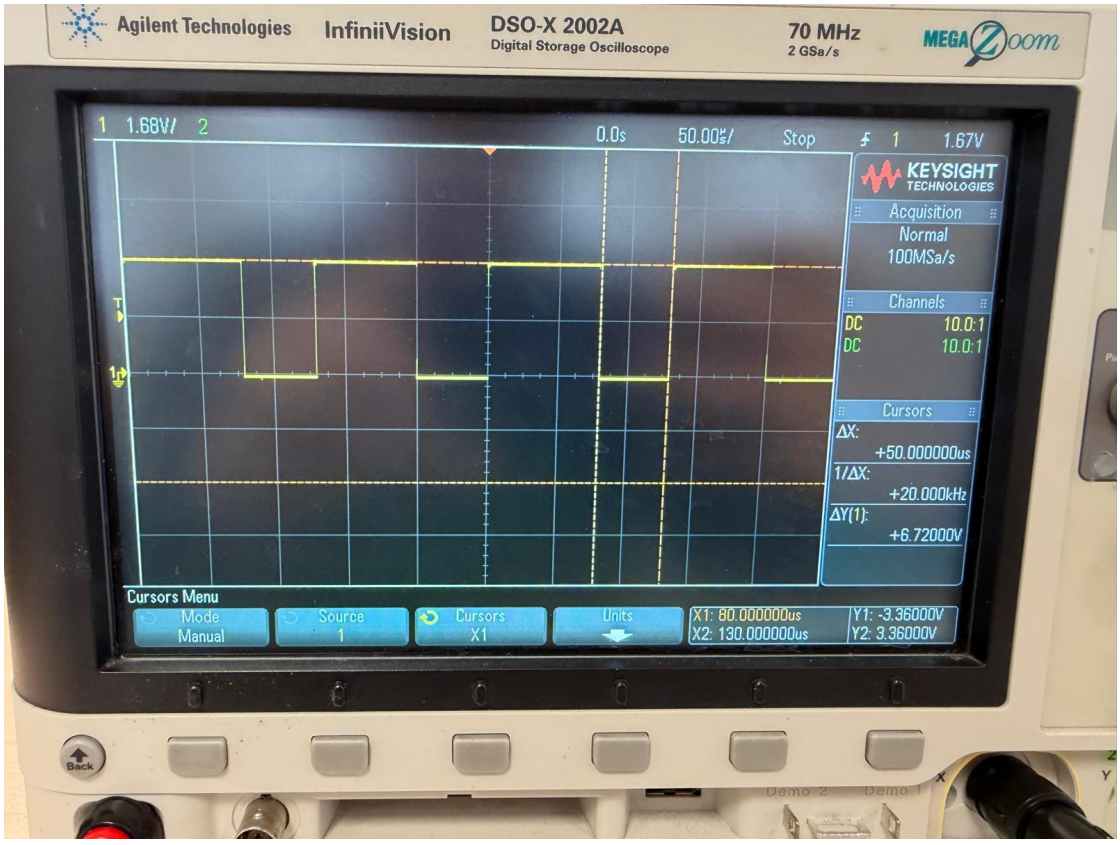


Figure 10: 50 µs Logic Low Wave measured on oscilloscope

To sum up the results, the timestamps from the behavioral simulation and the real measurements from the oscilloscope matched perfectly. This proves that the Basys3 board successfully generated the custom waveform with the exact time intervals we wanted. There were no unexpected delays or clock domain problems in the hardware, which verifies that the entire block design and VHDL implementation are fully functional.

Conclusion

In this lab I successfully designed an arbitrary waveform generator. Using the Clocking Wizard IP inside a block design was a new concept for me, but it is a very usefull way to manage different clock frequencies without writing custom dividers in VHDL. By doing both simulation and hardware tests, I observed that my calculated constants worked perfectly in real life. I also learned how to use the cursor feature on the oscilloscope to measure specific time intervals accurately. Overall it was a great experience that combined VHDL coding, IP integration and hardware measurement techniques.

Appendices

arbitrary_waveform.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity arbitrary_waveform is
  Port (
    clk_50MHz : in STD_LOGIC;
    reset_in  : in STD_LOGIC;
    wave_out  : out STD_LOGIC
  );
end arbitrary_waveform;

architecture Behavioral of arbitrary_waveform is

  type state_type is (S_HIGH_80, S_LOW_50_1, S_HIGH_70, S_LOW_50_2);
  signal current_state : state_type := S_HIGH_80;
```

```
  signal counter : integer range 0 to 4000 := 0;

  constant C_80US : integer := 4000;
  constant C_50US : integer := 2500;
  constant C_70US : integer := 3500;

begin

  process(clk_50MHz)
  begin
    if rising_edge(clk_50MHz) then
      if reset_in = '1' then
        current_state <= S_HIGH_80;
        counter <= 0;
        wave_out <= '0';
      else
        case current_state is
          when S_HIGH_80 =>
            wave_out <= '1';
            if counter < C_80US - 1 then
              counter <= counter + 1;
            else
              counter <= 0;
              current_state <= S_LOW_50_1;
            end if;

          when S_LOW_50_1 =>
            wave_out <= '0';
            if counter < C_50US - 1 then
```

```
                counter <= counter + 1;
              else
                counter <= 0;
                current_state <= S_HIGH_70;
              end if;

            when S_HIGH_70 =>
              wave_out <= '1';
              if counter < C_70US - 1 then
                counter <= counter + 1;
              else
                counter <= 0;
                current_state <= S_LOW_50_2;
              end if;

            when S_LOW_50_2 =>
              wave_out <= '0';
              if counter < C_50US - 1 then
                counter <= counter + 1;
              else
                counter <= 0;
                current_state <= S_HIGH_80;
              end if;
          end case;
        end if;
      end if;
    end process;

end Behavioral;
```

design_1_wrapper.vhd

```
`timescale 1 ps / 1 ps

module design_1_wrapper
  (clk_100MHz,
   reset_rtl_0,
   wave_out_0);
 input clk_100MHz;
 input reset_rtl_0;
 output wave_out_0;

 wire clk_100MHz;
 wire reset_rtl_0;
 wire wave_out_0;

 design_1 design_1_i
     (.clk_100MHz(clk_100MHz),
      .reset_rtl_0(reset_rtl_0),
      .wave_out_0(wave_out_0));
Endmodule
```

tb_design_1_wrapper.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_design_1_wrapper is
end tb_design_1_wrapper;

architecture Behavioral of tb_design_1_wrapper is
```

```
  component design_1_wrapper is
    port (
      reset_rtl_0 : in STD_LOGIC;
      clk_100MHz : in STD_LOGIC;
      wave_out_0 : out STD_LOGIC
    );
  end component;

  signal tb_clk : std_logic := '0';
  signal tb_reset : std_logic := '0';
  signal tb_wave_out : std_logic;

  constant C_CLK_PERIOD : time := 10 ns;

begin

  uut: design_1_wrapper port map (
    reset_rtl_0 => tb_reset,
    clk_100MHz => tb_clk,
    wave_out_0 => tb_wave_out
  );

  clk_process : process
  begin
    tb_clk <= '0';
    wait for C_CLK_PERIOD / 2;
    tb_clk <= '1';
    wait for C_CLK_PERIOD / 2;
  end process;
```

```
  stim_proc : process
  begin
    tb_reset <= '1';
    wait for 100 ns;
    tb_reset <= '0';
    wait for 1000 us;
    wait;
  end process;

end Behavioral;
```

Basys3.xdc

```
set_property PACKAGE_PIN W5 [get_ports clk_100MHz]
set_property IOSTANDARD LVCMOS33 [get_ports clk_100MHz]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk_100MHz]

set_property PACKAGE_PIN U18 [get_ports reset_rtl_0]
set_property IOSTANDARD LVCMOS33 [get_ports reset_rtl_0]

set_property PACKAGE_PIN J1 [get_ports wave_out_0]
set_property IOSTANDARD LVCMOS33 [get_ports wave_out_0]
```

References

[1] Bilkent University, Department of Electrical and Electronics Engineering. "EE102 Lab 6: Arbitrary Waveform Generator." EE102 Course Lab Manual, 2026.

[2] M. M. Mano and M. D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog*, 6th ed. Pearson, 2018.

[3] Digilent Inc., "Basys 3 FPGA Board Reference Manual." [Online]. Available: https://digilent.com/reference/programmable-logic/basys-3/reference-manual.

[4] Xilinx Inc., "Clocking Wizard LogiCORE IP Product Guide," UG047. [Online]. Available: https://docs.xilinx.com

[5] Wikipedia contributors, "Arbitrary waveform generator," Wikipedia, The Free Encyclopedia. [Online]. Available: https://en.wikipedia.org/wiki/Arbitrary_waveform_generator

[6] V. A. Pedroni, *Circuit Design and Simulation with VHDL*, 2nd ed. Cambridge, MA: MIT Press, 2010.

[7] Wikipedia contributors, "Finite-state machine," Wikipedia, The Free Encyclopedia. [Online]. Available: https://en.wikipedia.org/wiki/Finite-state_machine

[8] Keysight Technologies, "InfiniiVision 2000 X-Series Oscilloscopes User's Guide." [Online]. Available: https://www.keysight.com

[9] Wikipedia contributors, "Clock domain crossing," Wikipedia, The Free Encyclopedia. [Online]. Available: https://en.wikipedia.org/wiki/Clock_domain_crossing

[10] IEEE, "IEEE Standard VHDL Language Reference Manual," IEEE Std 1076-2008, 2009.